

# Дипломный проект

Тема проекта: "Музыкальный плеер с удаленной библиотекой и возможностью хранения связанных нот, табулатур"

Автор: Студент группы ВБД-321

Васильев Олег Олегович

Нижний Новгород, 2026

## Оглавление

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Введение.....</b>                                 | <b>3</b>  |
| <b>1. Краткий обзор решения.....</b>                 | <b>4</b>  |
| <b>2. Серверная часть.....</b>                       | <b>5</b>  |
| Внедрение зависимостей ASP.NET Core.....             | 6         |
| Контейнеризация серверного приложения.....           | 7         |
| Диаграмма классов серверного приложения.....         | 8         |
| <b>3. Структура базы данных.....</b>                 | <b>9</b>  |
| <b>4. Клиентская часть.....</b>                      | <b>11</b> |
| Клиентское приложение для компьютера.....            | 11        |
| <b>Диаграмма классов клиентского приложения.....</b> | <b>12</b> |
| MusicPro Common Library.....                         | 12        |
| MusicPro Desktop Application.....                    | 13        |
| <b>5. Диаграмма вариантов использования.....</b>     | <b>14</b> |
| <b>6. Исходный код приложений.....</b>               | <b>15</b> |
| Заключение.....                                      | 15        |

## Введение

Основная задача дипломной работы закрепить на практике изученные языки программирования и технологии. Вторичная цель создать полезное приложение для обучающихся музыке.

Музыкальных сервисов для прослушивания музыки довольно много. Сервисов для обучения музыке, тоже сотни. Но профессиональной коллекции музыкальных файлов и связанных с ними нот почему-то нет. Весьма удобно, обучаясь игре на музыкальном инструменте не тратить время на поиск, а просто слушая произведение сразу посмотреть ноты. Поставить на повтор и попробовать сыграть вместе с воспроизведенным отрывком.

А для музыкантов играющих в ансамбле или поющих в хоре важно, чтобы все заранее послушали одно и тоже произведение, а не какие-то разные его версии.

Именно поэтому тема данного проекта: “Музыкальный плеер с удаленной библиотекой и возможностью хранения связанных нот, табулатур”.

# 1. Краткий обзор решения

Решение состоит из двух компонентов:

1. Серверное приложение ASP.NET 9.0 на базе шаблона Core Web API с СУБД PostgreSQL

Разворачивается в Docker контейнере.

2. Кроссплатформенное клиентское десктопное приложение на JAVA FX.

Поддерживаемые ОС:

- Windows 10/11 (setup.exe)
- MacOS (DMG пакет)
- Linux редакции (DEB пакет) Debian, Ubuntu.

Серверное приложение подключается к серверу базы данных и взаимодействует с ней посредством Microsoft Entity Framework Core. Клиентское приложение обращается к серверу посредством HTTP запросов.

В рамках дипломного проекта реализована архитектура решения, и простейшая логика работы с музыкальными файлами и нотами. Получение информации с сервера, поиск/скачивание/прослушивание музыки, просмотр и скачивание связанных дополнительных файлов.

## 2. Серверная часть

Сервер независим от платформ клиентских приложений. API реализован в виде GET и POST запросов. Клиентами могут быть: сайты, мобильные приложения, десктопные приложения, а также сторонние серверные приложения.

Проект организован по упрощённой версии шаблона "Слойная архитектура" (Layered Architecture), характерной для ASP.NET Core приложений. Основные слои можно выделить по структуре каталогов:

**Объекты передачи данных (DTO/)** - объекты для обмена по HTTP

**Контроллеры (Controllers/)** - принимают HTTP-запросы, делегируют логику службам и возвращают ответы.

**Службы (Services/)** - служат связующим звеном между контроллерами и данными (UploadService, MusicService, ExtraFileService)

**Модели (Models/)** - представляют основные сущности приложения.

Модели характерные для профессиональной области:

- Музыкальный файл MusicFile
- Дополнительный файл ExtraFile
- Исполнитель Artist
- Жанр Genre
- Параметры поиска

Общие модели, могут использоваться в других проф. областях:

- Расширения файлов FileExtension
- Параметры пагинации PaginationParams

**Контекст данных (Database/)** - включает AppDbContext, который отвечает за взаимодействие с базой данных через Entity Framework Core.

## Внедрение зависимостей ASP.NET Core

За внедрение зависимостей отвечает встроенный контейнер зависимостей (DIContainer) самого фреймворка ASP.NET Core. Он является частью Microsoft.Extensions.DependencyInjection и настраивается в Program.cs через объект builder.Services (тип IServiceCollection). Регистрация зависимостей в контейнер выполняется в [Program.cs](#). Внутри классов внедрение происходит автоматически по аргументам конструктора.

Т.к теоретически кол-во поступающих запросов на сервер может быть очень большим важно организовать безопасную многопоточную обработку. Тип жизненного цикла объектов определяется архитектурным шаблоном.

Контроллеры по умолчанию создаются как Transient (новый экземпляр на каждый запрос).

Пользовательские сервисы (MusicService, ExtraFileService) - Scoped один раз на каждый HTTP-запрос.

AppDbContext - Scoped. Это стандартный и критически важный выбор. DbContext должен жить в рамках одного HTTP-запроса. Это обеспечивает изоляцию единицы работы, корректное управление подключениями к БД и предотвращает состояние гонки при параллельных запросах.

IUploadServiceFactory - Фабрика, как правило, не содержит состояния, специфичного для запроса, и не зависит от DbContext. Её задача — создавать другие объекты. Регистрация как Singleton эффективна: экземпляр создаётся один раз при старте приложения и используется всеми запросами, что снижает накладные расходы.

## Контейнеризация серверного приложения

Docker-контейнер — готовый к запуску включает в себя:

1. Само приложение: ASP.NET Core Web API (MusicServer.API), скомпилированный и готовый к работе.
2. Среду выполнения (.NET 9): Базовый образ `mcr.microsoft.com/dotnet/aspnet:9.0`, который предоставляет всё необходимое для запуска .NET-приложений в Linux.
3. Системные зависимости: В контейнер добавлены библиотеки (`libgdiplus`, `libc6-dev`), которые необходимы библиотеке `TagLibSharp` для корректной работы с аудиофайлами в Linux.
4. Структуру для данных: Внутри контейнера созданы директории `/app/data/music` и `/app/data/extra`, куда будут сохраняться загруженные файлы.
5. Сетевые настройки: Контейнер «слушает» порты 5098 (основной API) и 7044, которые вы можете пробросить на порты вашего компьютера или сервера.

**Главная ценность этого контейнера в том, что приложение будет работать одинаково на любом компьютере или в облаке, где установлен Docker.**

### Как запустить контейнер:

1. Установите Docker Desktop на свой компьютер.
2. Клонировать репозиторий: `git clone https://github.com/vaClev/MusicPlayer.git`
3. Перейдите в папку проекта и создайте файл окружения `.env` с паролем для базы данных.
4. Запустите контейнеры командой из папки `Server`:

```
docker-compose up -d
```

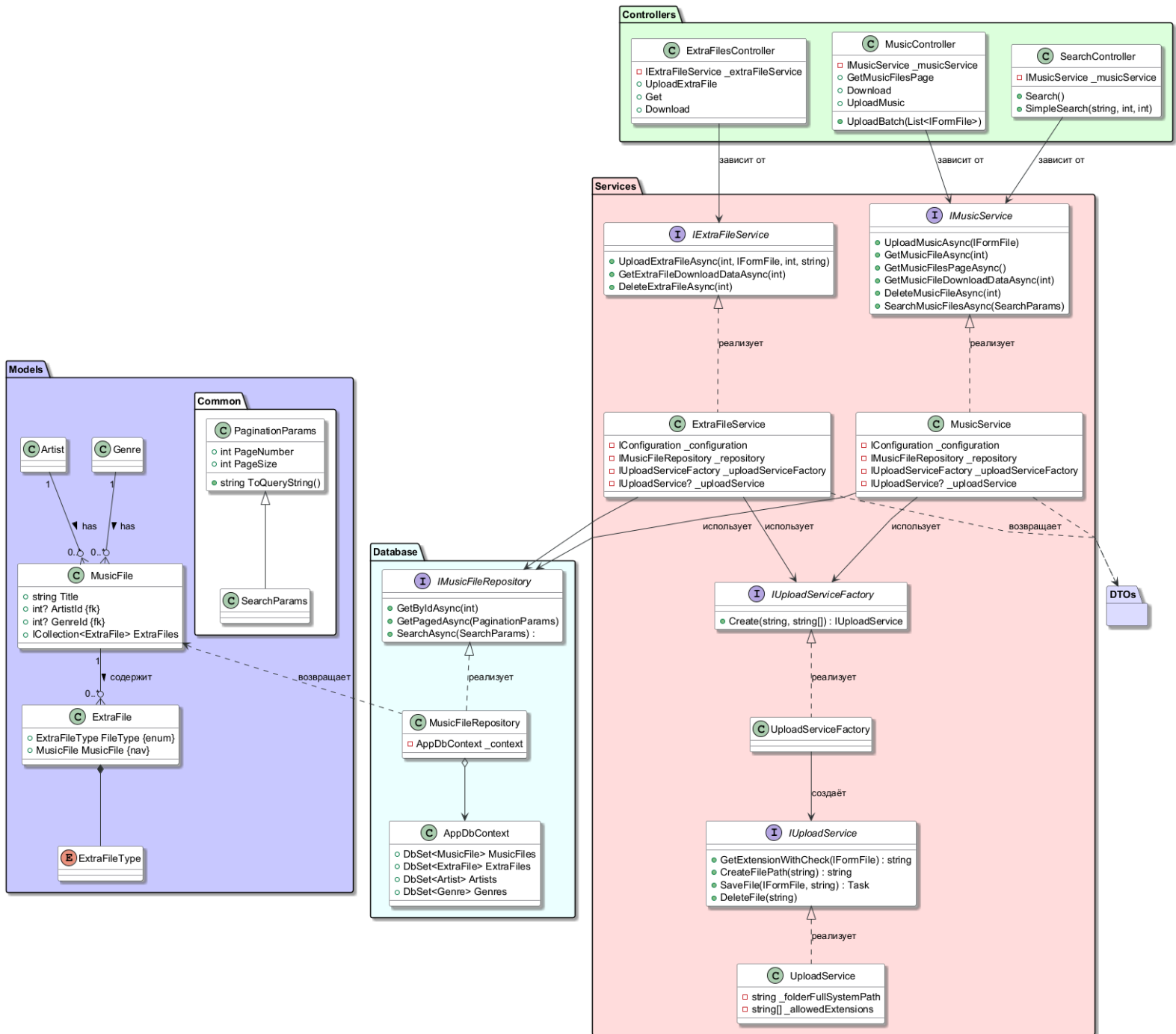
Docker скачает нужные образы, соберет ваш сервер и запустит его вместе с PostgreSQL.

5. Проверьте работу, открыв в браузере `http://localhost:5098/api/music`.

Полная инструкция: [github.com/vaClev/MusicPlayer/tree/main/Server/Docker%20copies](https://github.com/vaClev/MusicPlayer/tree/main/Server/Docker%20copies)

# Диаграмма классов серверного приложения

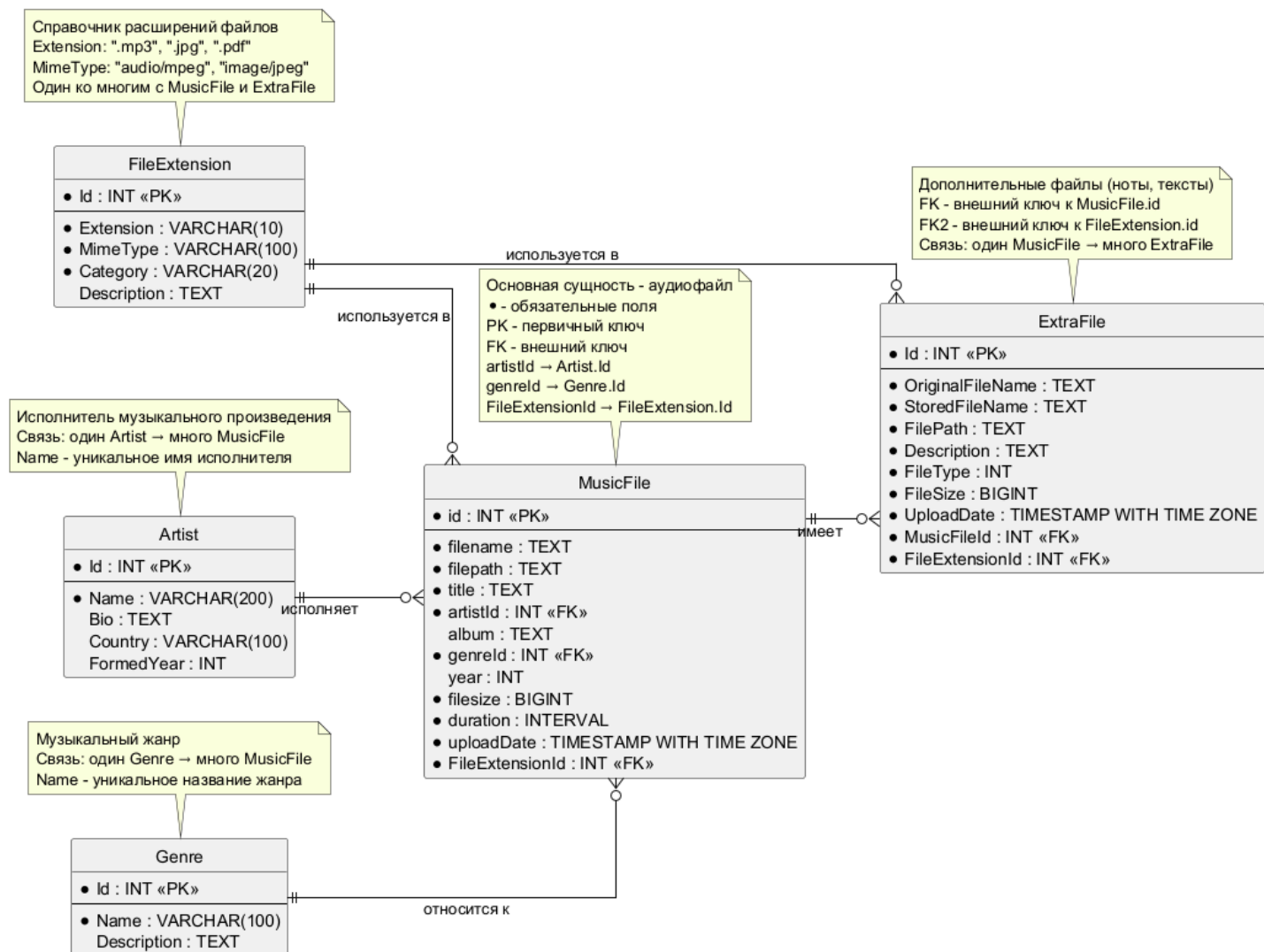
Полноразмерная диаграмма доступна по ссылке: <https://raw.githubusercontent.com/vaClev/>



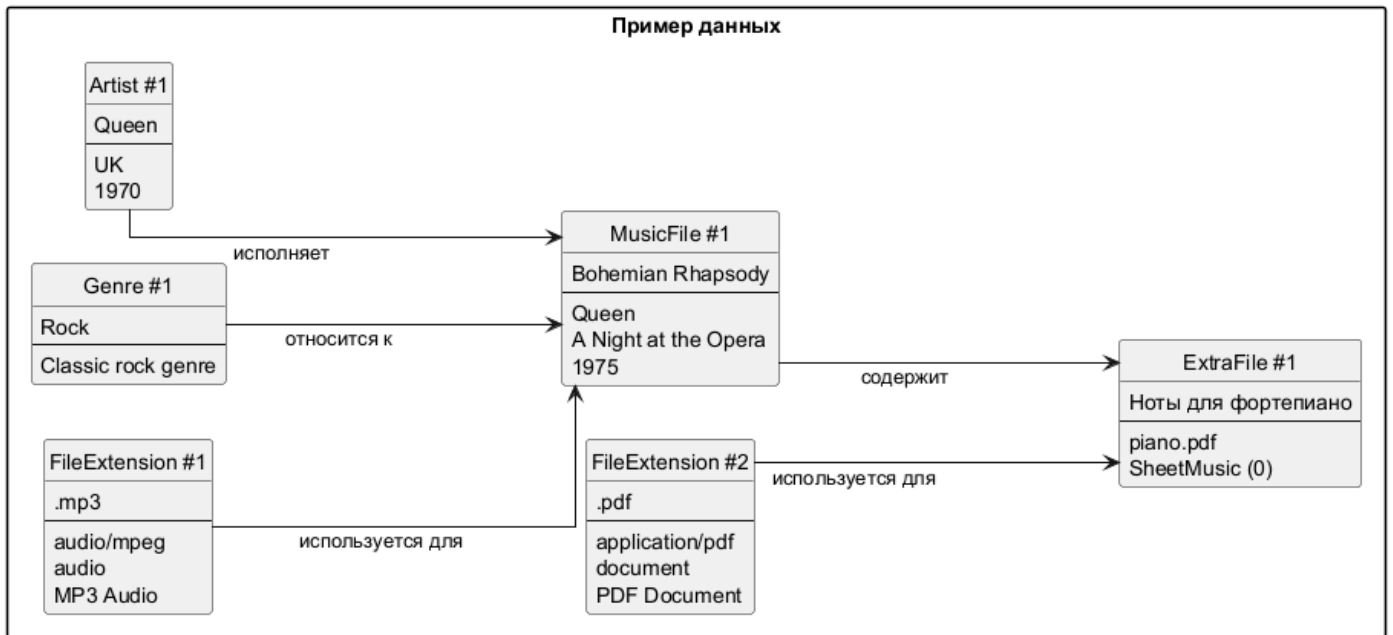
### 3. Структура базы данных

Сущности (MusicFile, ExtraFile) требуют реляционной структуры с чёткими связями. Важна поддержка каскадного удаления (удалил трек — автоматически удалились связанные ноты). Также пользователю часто необходимо легко найти одну сущность по известной другой.

Фактически с этой задачей может справиться любая современная СУБД. PostgreSQL выбран для учебного проекта, т.к он свободный и открытый, и простой в развертывании.



Пример данных хранимых в БД:



## 4. Клиентская часть

Клиентское-приложение состоит из двух компонентов :

### 1. Библиотека классов MusicPro Common Library:

- ☐ **Объекты передачи данных (dto/)** - объекты для обмена по HTTP. Объекты парсинга JSON ответов сервера.
- ☐ **Модели (Models/)** - основные сущности приложения
- ☐ **Службы (services/)** - по сути выполняют всю работу, возвращают результат контроллерам. (DownloadService, IPlayerService, MusicClientService)

### 2. Приложение для компьютеров MusicPro Desktop Application

- ☐ **Контроллеры (controllers/)** - контроллеры UI элементов
- ☐ **UI Формы (views/)** - FXML файлы разметки для интерфейса.
- ☐ **Служба BasicPlayerService** - реализация плеера на JavaFX MediaPlayer

Такое разделение позволяет переиспользовать код на разных операционных системах. Например, возможно создать Android приложение и подключить к нему библиотеку *"MusicPro Common Library"*.

## Клиентское приложение для компьютера

Клиентское приложение для компьютера хотелось сделать кроссплатформенным, поэтому в качестве UI выбрана JAVA FX.

Для запуска необходима поддержка операционной системой JAVA OpenJDK 21U и JAVA FX version 21.0.9. В рамках дипломного проекта проверена собираемость и запуск на трех ОС:

- Windows 11
- Ubuntu 24.04
- MacOS Monterey 12.5

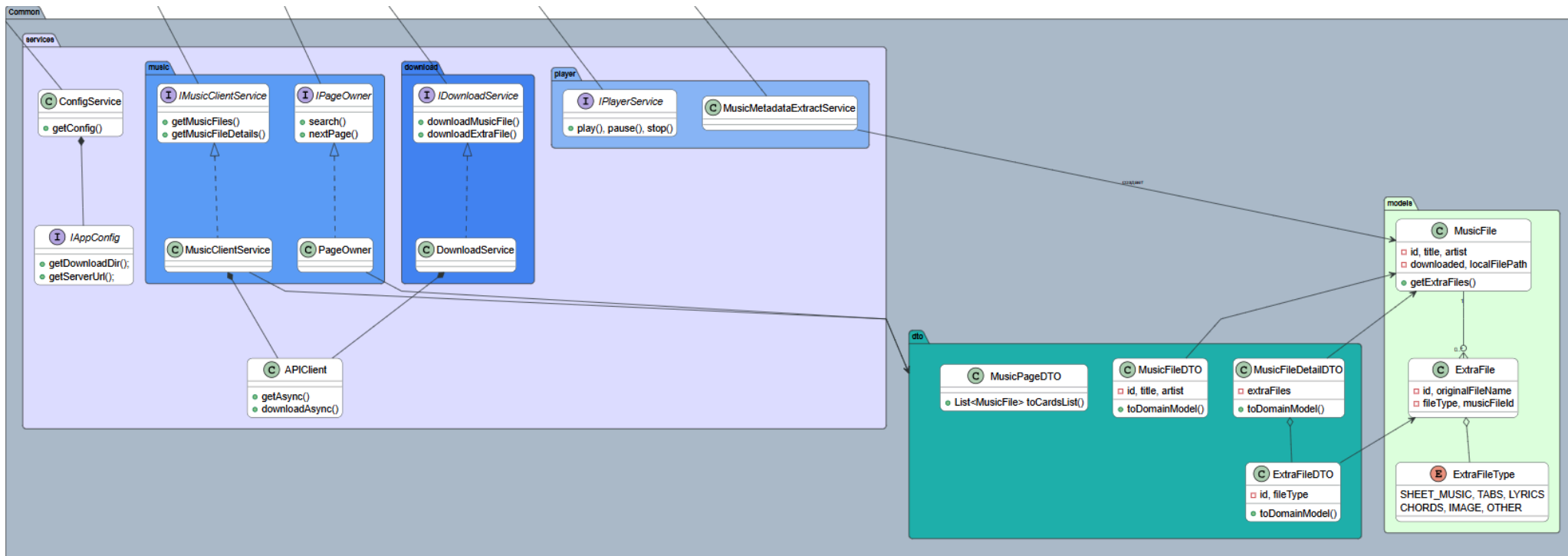
Инструкции по сборке установочных пакетов в соответствующих папках:

<https://github.com/vaClev/MusicPlayer/tree/main/Client>

## Диаграмма классов клиентского приложения

Полноразмерная диаграмма классов доступна по ссылке: <https://raw.githubusercontent.com/vaClev/MusicPlayer/.../ClientSimpleDiagram.png>

### MusicPro Common Library



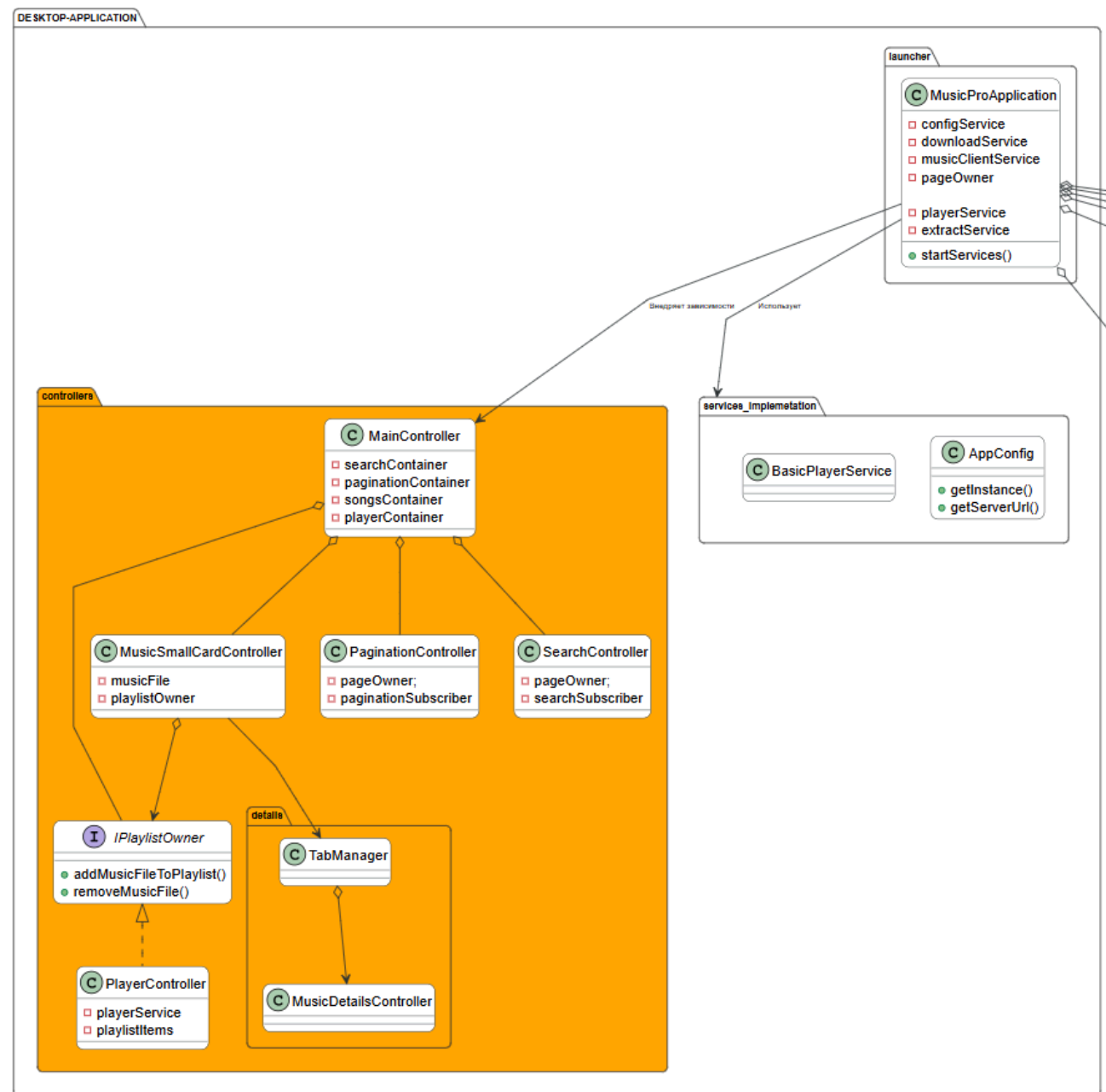
MusicPro Common Library использует две сторонние библиотеки:

- `com.google.gson.Gson` – парсинг JSON.
- `org.jaudiotagger` – извлечение метаданных из mp3

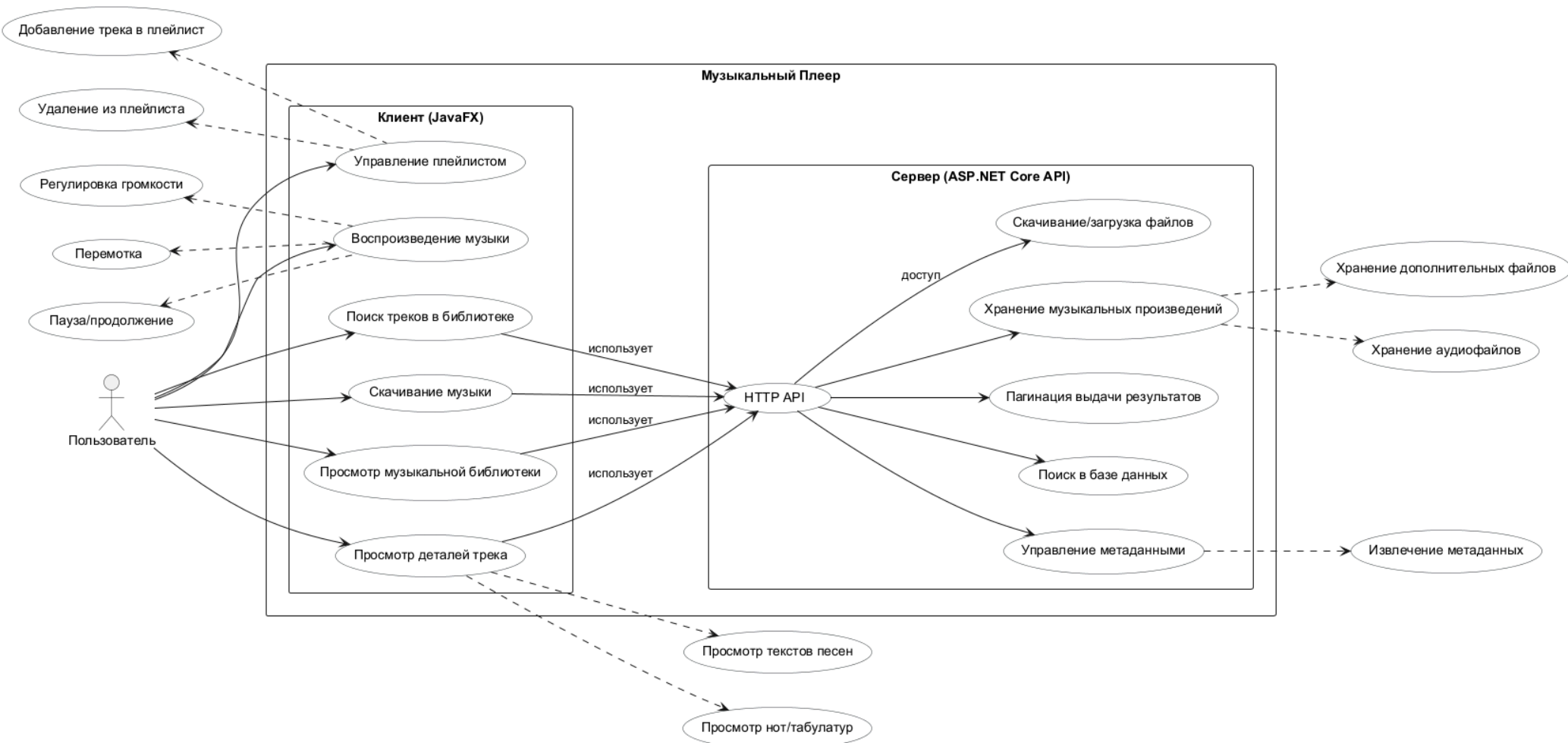
## MusicPro Desktop Application

Зависит от библиотек:

- ☐ MusicPro Common Library
- ☐ JAVA FX 21.0.9



## 5. Диаграмма вариантов использования



## 6. Исходный код приложений

Исходный код доступен по ссылке: <https://github.com/vaClev/MusicPlayer>

### Заключение

Для меня основной целью работы было изучить архитектуру клиент-серверных приложений и создать прототип приложения. Эта цель достигнута.

Итогом работы является простое с точки зрения конечного пользователя приложение, но его архитектура открыта для расширения и наращивания функциональности.

Серверное приложение, может быть легко адаптировано под хранение и скачивание любых файлов, например текстовых, графических документов, фото.